Interview Preparation Guide

Durai Pandi | .NET Software Engineer | Based on your resume (Resulticks & Agilensmart Technology Solutions)

CORE .NET & PROGRAMMING

Q1. Walk me through your experience with .NET Core vs .NET Framework. When would you choose one over the other?

ANSWER

I explain that .NET Core (now unified as .NET) is cross-platform, lightweight, and better suited for modern, cloud-native or Linux-hosted applications, while .NET Framework is Windows-only and typically used for legacy enterprise apps. I choose based on hosting environment, performance needs, and whether the project needs cross-platform deployment.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

At Resulticks, our communication platform runs on Linux-hosted production servers, which directly influenced decisions around library and framework choices — for example, migrating our mail-sending library to one that performs reliably in that cross-platform environment.

Q2. You've worked with both Entity Framework and Dapper. What's the difference, and how do you decide which to use?

ANSWER

Entity Framework is a full ORM that's great for rapid development, built-in change tracking, and complex object relationships, but it adds overhead. Dapper is a lightweight micro-ORM that gives near-raw-SQL performance with less abstraction, which I prefer for high-throughput or performance-critical queries.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

In my work integrating with systems like Amazon SP-API and KEEPA API, where I needed to retrieve and refresh large volumes of product/pricing records efficiently, a lightweight data-access approach like Dapper helps keep database round-trips fast compared to a heavier ORM.

Q3. How do you approach debugging a production issue in a live .NET application?

ANSWER

I start by reproducing the issue or reviewing logs/error traces to isolate where it's occurring, check recent deployments or config changes as likely causes, form a hypothesis, and validate it in a safe environment before pushing a fix. I prioritize minimizing customer impact while I investigate.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

At Resulticks, I regularly investigated client-reported issues and production bugs on our communication platform, tracing them back to root cause and deploying fixes directly to Linux production servers while coordinating releases to keep downtime minimal.

EMAIL, MESSAGING & SMTP

Q4. You migrated your mail-sending library from EASendMail to MailKit. Why was that necessary, and what was the impact?

ANSWER

I explain the business driver first — reliability and delivery performance — then the technical reasoning: MailKit is open-source, actively maintained, cross-platform, and integrates well with Linux-hosted environments, whereas the

older library had compatibility and performance limitations in that setup.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

This was a real project at Resulticks. Migrating from EASendMail to MailKit directly improved our email delivery success rate and sending speed on our Linux-hosted production servers — a concrete, measurable improvement I can speak to with before/after context.

Q5. What is SMTP domain automation, and why does it matter for a communication platform?

ANSWER

It's the process of automating the setup and configuration of sending domains (like SPF, DKIM, DNS records) so outbound email is properly authenticated and deliverable, without requiring manual setup for every new client or domain.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

I designed and built SMTP domain automation at Resulticks specifically to reduce the manual effort involved in configuring outbound mail domains, which streamlined onboarding and reduced errors compared to doing it by hand for each client.

Q6. How would you troubleshoot poor email deliverability in a high-volume messaging system?

ANSWER

I'd check authentication records (SPF/DKIM/DMARC), review bounce/error logs from the SMTP provider, confirm the sending library is configured correctly, and look at sender reputation. I'd also check for rate-limiting or throttling issues under high volume.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

This mirrors real work I did at Resulticks — investigating client-reported delivery issues and resolving them with minimal disruption to the transactional and marketing messages our platform sends via email and SMS.

API INTEGRATION & AMAZON SELLER TOOLS

Q7. Tell me about a project where you integrated a third-party API. What challenges did you face?

ANSWER

I'd describe the goal of the integration, how I handled authentication, rate limits, and error handling, and how I structured the data once retrieved so it was usable downstream.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

I built a product history tracking system that takes JSON queries or ASINs from user input, calls the KEEPA API to fetch historical product data, and stores the results in Google Sheets via the Google Sheets API for the team to analyze and report on.

Q8. Describe your experience working with the Amazon SP-API. What kind of business problem did it solve?

ANSWER

I'd explain that SP-API lets sellers programmatically manage inventory, pricing, and listings, and that I used it to keep internal systems in sync with live marketplace data.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

I built a stock and price management tool integrated with Amazon SP-API that retrieved database records and refreshed stock status and pricing from live product data, plus a competitor listing monitor that flagged identical products and generated feed files for upload to Amazon Seller Central.

Q9. How do you design a system that needs to process bulk data efficiently, like your wholesale analyzer tool?

ANSWER

I'd focus on batching requests, minimizing redundant calls to external APIs, using efficient data access (e.g., Dapper for fast queries), and structuring the pipeline so it can scale to larger product catalogs without timing out.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

I developed wholesale bulk analysis software for Amazon FBA and merchant-fulfilled sellers, giving them a fast way to identify profitable products across large catalogs — this was one of three connected Amazon-seller tools I built using a consistent C#/.NET approach.

BEHAVIORAL & EXPERIENCE-BASED

Q10. Tell me about a time you improved system performance or reliability.

ANSWER

I'd use the STAR method — Situation, Task, Action, Result — and lead with the measurable outcome.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

Situation: our platform's email delivery relied on an older library with reliability issues on Linux servers. Task: improve delivery success and speed. Action: I migrated the mail-sending library from EASendMail to MailKit. Result: measurable improvement in email delivery success rate and sending speed in production.

Q11. Describe a situation where you had to resolve a production issue under pressure.

ANSWER

I'd walk through how I stayed methodical — isolating the issue, communicating status, and deploying a fix without causing further disruption.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

At Resulticks, I investigated client-reported issues and production bugs on a live customer communication platform, then deployed fixes directly to Linux production servers while coordinating releases to keep downtime to a minimum.

Q12. How do you keep your code and applications up to date with the latest frameworks and libraries?

ANSWER

I explain that I treat this as ongoing technical debt management — reviewing changelogs, testing upgrades in lower environments first, and rolling out incrementally.

REAL-TIME EXAMPLE (FROM YOUR EXPERIENCE)

I regularly enhanced and refactored existing .NET application functionality at Resulticks, upgrading code to align with the latest framework and library versions as part of normal maintenance, not just as a one-off project.

Tip: Practice saying each 'Real-Time Example' out loud in the STAR format (Situation, Task, Action, Result) — interviewers respond well to specific, measurable outcomes over general descriptions.